

成绩	
评阅人	
日期	2024 年 月 号

重庆邮电大学

学生实验实习报告册

学年学期： 2023-2024 学年 ☐ 春 ☒ 秋学期

课程名称： 人工智能与大数据课程设计

学生学院： 通信与信息工程学院

专业班级： 01012205

学生学号： 2022210410

学生姓名： 雷宇航

联系电话： 18580086873

重庆邮电大学教务处制

课程名称	人工智能与大数据课程设计	课程编号	A2012050
实验地点	YF317	实验时间	2024.12.27
指导教师	胡林		
实验名称	Python 聚类决策树训练与预测		
评阅人签字		成绩	

一、实验目的

- 1、掌握决策树的基本原理并理解监督学习的基本思想。
- 2、掌握 Python 实现决策树的方法。

二、实验原理

- 1、运用 Anaconda 搭建的 Jupyter Notebook 平台编写实例 Python 程序。
- 2、利用 Python 实现决策树，构建分类模型。
- 3、编写代码，调试程序，完成实验报告。

三、使用软件平台

- 1、Windows 11 电脑一台。
- 2、Python、Anaconda、Jupyter Notebook 平台。

四、实验步骤、代码、结果及结果分析（复制代码，结果截图）

1. 实例 1：基于 ONE-HOT 编码的银行贷款决策数

代码：

```
1. ## 实例一
2. import pandas as pd
3. import sklearn as sklearn
4. import os
5. os.environ["PATH"] += os.pathsep + "C:\\Program Files\\Graphviz\\bin"
6.
7. # 引入字典特征提取 DictVectorizer 类，实现从特征值到向量的映射。向量表示为 ONE-HOT 编码
8. from sklearn.feature_extraction import DictVectorizer
9.
10. # 引入 sklearn 库中的树相关 tree 模块
11. from sklearn import tree
12. # pandas 读取 csv 文件, header = None 表示不将首行作为标题, 无 header 参数,
```

则将默认将首行作为标题

```
13.data = pd.read_csv('loan_YN.csv',encoding='gbk')
14.print(data)
15.
16.# 指定列名
17.data.columns = ['age', 'income', 'guding','VIP','loan']
18.
19.# 生成特征提取对象 vec, sparse 参数的意思是否产生稀疏矩阵, False 表示不产生
20.vec = DictVectorizer(sparse=False)
21.# 提取数据的输入列为 feature (输入特征)
22.feature = data[['age', 'income', 'guding','VIP']]
23.#print(feature)
24.# 先用 pandas 的.to_dict() 函数对 feature 生成字典,orient 参数表示字典构建方式, 'record' 表示对每一行生成字典
25.# 然后基于 vec 对象的 fit_transform() 函数进行向量化, 得到 ONE-HOT 编码的向量
26.#14 个样本, 每个特征包含的状态为 3 (青年中年壮年)、3 (高中低)、2 (是否)、2 (是否), 所以有 10 个比特
27.X_train = vec.fit_transform(feature.to_dict(orient='record'))
28.#print(X_train)
29.# 打印各个变量, vec.get_feature_names() 显示 01 编码后, 每列的属性名, 如 VIP=否, 是第一列
30.print('show feature\n', feature)
31.print('show vector\n', X_train)
32.print('show vector name\n', vec.get_feature_names())
33.print('show vector name\n', vec.vocabulary_)
34.
35.# 提取数据的输出列为输出数据 Y_train
36.Y_train = data['loan']
37.
38.# 基于 tree 模块中的 DecisionTreeClassifier 类, 生成决策树对象 clf
39.# 参数 criterion='entropy' 表示基于信息熵生成决策树
40.clf = tree.DecisionTreeClassifier(criterion='entropy')
41.
42.# 根据向量化输入特征和输出数据, 训练决策树 clf
43.clf.fit(X_train, Y_train)
44.
45.# 导入绘图库, #Graphviz 点语言的一个 python 接口
46.import pydotplus
47.import graphviz
48.
49.# 用于输出图片中的中文正常显示
50.from six import StringIO
51.
52.# 用于显示图片
53.from IPython.display import Image
54.
```

```

55.dot_data = StringIO()
56.
57.# export_graphviz()函数生成决策树的graphviz 表示, 导出为dot 格式
58.# 参数说明: clf 即为训练的决策树; out_file 表示可视化结果的写入位置;
    feature_names 表示特征名称
59.tree.export_graphviz(clf, out_file =dot_data,feature_names = vec.get
    _feature_names(), rounded=True)
60.
61.# 替换中文字体显示, 将西文字体'helvetica' 替换成中文"Microsoft YaHei"
62.graph = pydotplus.graph_from_dot_data(dot_data.getvalue().replace('h
    elvetica','"Microsoft YaHei"'))
63.
64.# 导出并显示决策树
65.graph.write_pdf("test_071.pdf")    #保存图片为pdf 格式
66.Image(graph.create_png())    #显示图片
67.
68.## 例1
69.result = clf.predict([[1., 0., 0., 0., 1., 1., 0., 0., 0., 1.],\
70.                        [0., 1., 0., 0., 1., 1., 0., 0., 0., 1.],\
71.                        [1., 0., 1., 0., 0., 1., 0., 0., 0., 1.]])    #
    这个是训练数据集的前三个
72.print(result)
73.## 例2
74.#用户甲, 年龄 28 岁、有固定收入、不是 VIP 会员。决策结果: 提供贷款
75.result = clf.predict([[ 1., 0., 0., 0., 1., 0., 1., 0., 0., 0.]])
76.print(result)
77.#用户乙, 年龄 51 岁、无固定收入、是 VIP 会员。决策结果: 不提供贷款
78.result = clf.predict([[ 0., 1., 0., 1., 0., 1., 0., 0., 0., 0.]])
79.print(result)

```

输出:

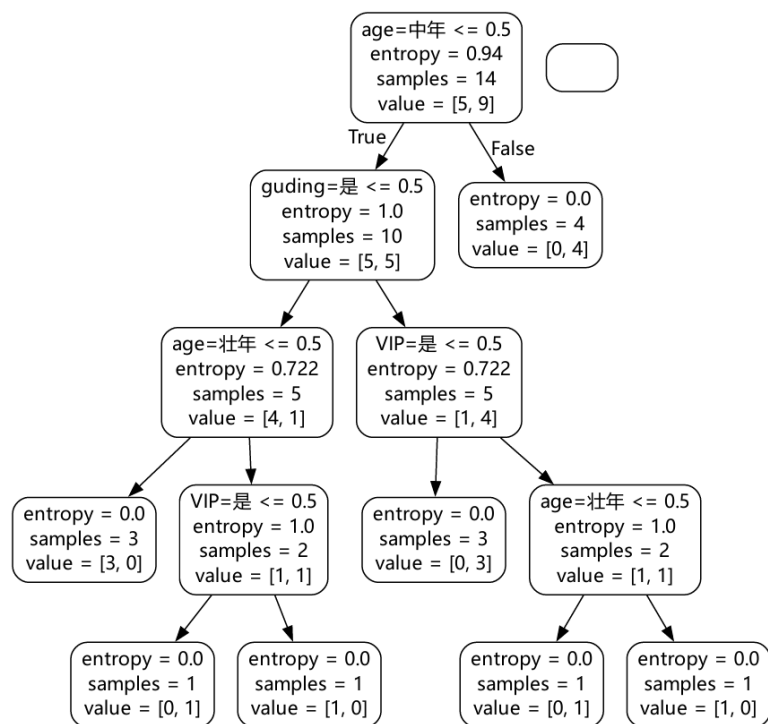


图 1 银行贷款决策树

2. 思考 1：基于特征数字化的银行贷款决策树

代码：

```

1. ## 思考一
2. import pandas as pd
3.
4. # 引入 sklearn 库中的树相关 tree 模块
5. from sklearn import tree
6.
7. data = pd.read_csv('loan_YN_01.csv', encoding='gbk')
8.
9. # 指定列名
10. data.columns = ['age', 'income', 'guding', 'VIP', 'loan']
11.
12. # 根据列名，分别提取输入数据 X_train 和输出数据 Y_train
13. X_train = data[['age', 'income', 'guding', 'VIP']]
14. Y_train = data['loan']
15.
16. # 基于 tree 模块中的 DecisionTreeClassifier 类，生成决策树对象 clf
17. # 参数 criterion='entropy' 表示基于信息熵生成决策树
18. clf = tree.DecisionTreeClassifier(criterion='entropy')
19.
20. # 根据输入数据和输出数据，训练决策树 clf
21. clf.fit(X_train, Y_train)
22.
23. ###
  
```

```

24. # 导入绘图库
25. import pydotplus
26.
27. # 用于输出图片中的中文正常显示
28. from six import StringIO
29.
30. # 用于显示图片
31. from IPython.display import Image
32.
33. dot_data = StringIO()      #在内存中读写字符串，这里主要是写
34.
35. # export_graphviz()函数生成决策树的graphviz 表示，导出为dot 格式
36. # 参数说明: clf 即为训练的决策树; out_file 表示可视化结果的写入位置;
    feature_names 表示特征名称
37. tree.export_graphviz(clf, out_file = dot_data, feature_names = ['age'
    , 'income', 'guding', 'VIP'])
38.
39. # 根据生成的dot 格式数据，渲染可视化图形
40. graph = pydotplus.graph_from_dot_data(dot_data.getvalue())    #getvalue()方法用于获得写入的字符串，中文乱码
41.
42. graph.write_pdf("test_072.pdf")    #保存图片为pdf 格式
43. Image(graph.create_png())    #显示图片
44.
45. ###
46. ## 例1
47. result = clf.predict([[ 0., 0., 0., 0.],\
48.                        [ 0., 0., 0., 1.],\
49.                        [1., 0., 0., 0.]])    #这个是训练数据集的前三个
50. print(result)
51. ## 例2
52. #用户甲，年龄 28 岁、有固定收入、不是 VIP 会员。决策结果：提供贷款
53. result = clf.predict([[ 0., 0., 1., 0.]])
54. print(result)
55. result = clf.predict([[ 0., 1., 1., 0.]])
56. print(result)
57. #用户乙，年龄 51 岁、无固定收入、是 VIP 会员。决策结果：不提供贷款
58. result = clf.predict([[ 2., 0., 0., 1.]])
59. print(result)
60. result = clf.predict([[ 2., 1., 0., 1.]])
61. print(result)

```

输出：

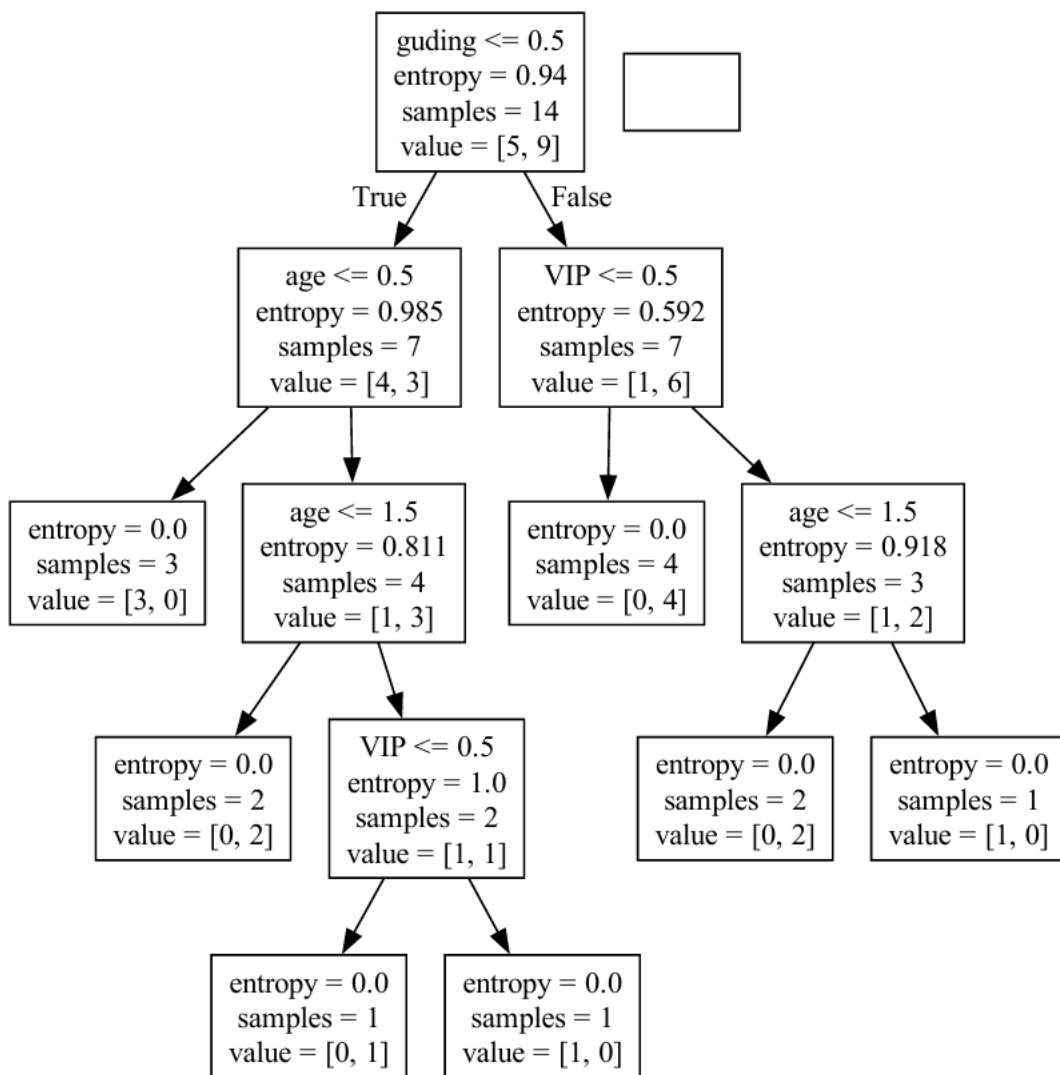


图 2 基于特征数字化的银行贷款决策树

五、碰到的问题及解决办法

在实例 1 中，当输出决策树时遇到中文乱码问题，若直接使用包含中文的代码会报错。通过查阅资料，我发现将原本一行中的替换操作与图形生成分开为两行代码可以解决此问题。即先替换字体为“Microsoft YaHei”，然后再用处理后的数据创建图形。修改后，决策树中的中文能正常显示，不再乱码。

六、实验心得体会

通过本次实验，我深入理解了决策树的基本原理与监督学习的核心思想。同时，我学会了使用 Python 实现决策树的方法，进一步加深了对 Pandas 库、sklearn 库以及字典的理解与应用。